toetsLib

AI-Powered Playwright Test Generator

Version 0.1.0 — User Manual

"toets" is Afrikaans for "test"

Point it at your React repo. Get a full Playwright test suite.
With screenshots, page objects, and a diagnostic report.

Table of Contents

1 Introduction

2 Installation

3 Quick Start

4 How It Works — The 5-Stage Pipeline

5 Commands Reference

6 Configuration

7 Authentication

8 Generated Output

9 Diagnostic Reports

10 Supported Frameworks

11 Real-World Example: Lefa Pay

12 CI/CD Integration

13 Troubleshooting

14 Architecture

1. Introduction

toetsLib is an AI-powered CLI tool that analyzes React repositories and automatically generates comprehensive Playwright end-to-end test suites.

Instead of writing tests by hand, you point toetsLib at your project and it:

- Parses your source code to understand routes, forms, API calls, and auth patterns
- Starts your app and crawls it with a headless browser to discover real UI elements
- Uses Claude AI to generate Playwright tests with Page Object Models
- Runs the generated tests and produces a diagnostic report with fix instructions

The output is a complete `e2e/` directory with `playwright.config.ts`, page objects, fixtures, test specs, and a `DIAGNOSTIC.md` that tells your team exactly what to fix if any tests fail.

2. Installation

Prerequisites

- **Node.js** 18 or later
- **npm** (or yarn, pnpm, bun)
- **ANTHROPIC_API_KEY** environment variable — required for test generation

Install from Source

```
git clone <repo-url> toetslib
cd toetslib
npm install
npx turbo build
```

Run the CLI

```
# Direct execution
node apps/cli/dist/index.js --help

# Or link globally
cd apps/cli && npm link
toetslib --help
```

Set Your API Key

```
export ANTHROPIC_API_KEY="sk-ant-api03-..."
```

> **Note:** The `analyze` and `crawl` commands work without an API key since they don't use AI. Only `generate` and `validate` require it.

3. Quick Start

Generate a full Playwright test suite in one command:

```
toetslib generate ./my-react-app
```

That's it. Your tests land in `./e2e/` by default.

Step-by-Step First Run

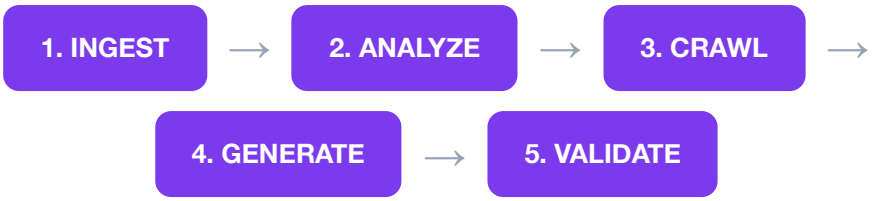
```
# 1. Check what toetsLib detects in your project
toetslib analyze ./my-app --json

# 2. See what the browser finds (starts dev server)
toetslib crawl ./my-app --verbose

# 3. Generate tests without running them
toetslib generate ./my-app --no-validate

# 4. Generate tests AND run them with diagnostics
toetslib generate ./my-app --verbose
```

4. How It Works — The 5-Stage Pipeline



Stage 1: Ingest

Reads your project and detects its configuration automatically.

Detection	Method
Package manager	Lock file check (package-lock.json → npm, yarn.lock → yarn, etc.)
Framework	Dependencies in package.json (vite + plugin-react → react-vite, next → nextjs)
Dev command	Reads `scripts.dev` or `scripts.start`
Dev port	Parsed from vite.config.js / next.config.js
Source directory	Checks for src/, app/, pages/
Workspace root	Walks up directory tree to find workspace package.json

Stage 2: Analyze (Static Code Analysis)

Parses your source files using Babel AST to understand the application structure.

- **Routes:** React Router declarations, route arrays, constant references, file-based routing
- **Components:** Maps which page components render on which routes, including child components
- **Forms:** Detects form fields from both native HTML elements and custom components (InputField, MoneyInput, SelectField, etc.)

- **API calls:** Extracts endpoints from axios, fetch, TanStack Query hooks (useQuery, useMutation)
- **Auth patterns:** Identifies auth contexts, route guards, login redirects, and trial/guest modes

Stage 3: Crawl (Live Browser)

Starts your dev server and visits your app with a headless Playwright browser.

- Visits every discovered route
- Takes screenshots at 3 viewport sizes (mobile 375px, tablet 768px, desktop 1280px)
- Finds all interactive elements with their selectors
- Records the navigation graph (which pages link to which)
- Logs console errors and failed network requests
- Handles login with provided credentials (if configured)

Stage 4: Generate (AI-Powered)

Uses Claude AI to produce Playwright test code in three passes:

Pass	Input	Output
1. Test Plan	Full AppMap summary	Structured JSON: which pages get tests, what categories, test names
2. Page Objects	Per-page: crawl data + screenshot + form fields	TypeScript Page Object Model (.page.ts) per page
3. Test Code	Test plan + page object code	Playwright spec files (.spec.ts) using page objects

> **Screenshots are sent to the AI** so it can "see" each page while writing tests. This produces much better selectors than text-only analysis.

Stage 5: Validate (Diagnostic)

Runs the generated tests and produces a diagnostic report.

- Executes all tests via Playwright
- Parses failures by category: selector, timeout, assertion, navigation
- Writes `DIAGNOSTIC.md` with step-by-step fix instructions per failure
- Writes `.toetslib-report.json` for machine consumption

5. Commands Reference

generate

Full pipeline: analyze, crawl, generate tests, and validate them.

```
toetslib generate <repo-path> [options]
```

Option	Default	Description
`-o, --output <dir>`	`./e2e`	Output directory for generated tests
`--auth-url <url>`	—	Login page URL for authenticated testing
`--auth-email <email>`	—	Test account email
`--auth-password <pass>`	—	Test account password
`--config <path>`	`.toetslib.json`	Path to config file
`--max-heal-cycles <n>`	3	Max diagnostic cycles
`--no-validate`	false	Skip validation (generate only)
`--verbose`	false	Detailed debug output
`--json`	false	Output full AppMap as JSON

analyze

Static analysis only. Shows what toetsLib detects without starting the app.

```
toetslib analyze <repo-path> [--json] [--verbose] [--conf
```

crawl

Analyze + crawl. Starts the app and visits pages with a headless browser.

```
toetslib crawl <repo-path> [--json] [--verbose] [--auth-u
```

validate

Run existing tests and produce a diagnostic report.

```
toetslib validate <repo-path> [-o <dir>] [--verbose] [--j
```

6. Configuration

Create a `.toetslib.json` or `toetslib.json` in your project root:

```
{
  "devCommand": "npm run dev",
  "devPort": 5173,
  "baseUrl": "http://localhost:5173",
  "auth": {
    "strategy": "form",
    "loginUrl": "/login",
    "fields": {
      "email": "test@example.com",
      "password": "secret"
    },
    "submitSelector": "button[type=submit]"
  },
  "exclude": ["/admin/*"],
  "outputDir": "./e2e",
  "maxHealCycles": 3
}
```

Field	Type	Description
`devCommand`	string	Dev server start command. Auto-detected if not set.
`devPort`	number	Dev server port. Auto-detected from config files.
`baseUrl`	string	Full base URL. Built from port if not set.
`auth`	object	Authentication config (see Section 7).
`exclude`	string[]	Route patterns to skip (glob syntax).
`outputDir`	string	Where to write tests. Default: `./e2e`
`maxHealCycles`	number	Diagnostic retry cycles. Default: 3

Priority Order

When the same setting appears in multiple places:

1. **CLI flags** (highest)
2. **Config file** (.toetslib.json / toetslib.json)
3. **Auto-detected values** (lowest)

7. Authentication

Via CLI Flags

```
toetslib generate ./my-app \
  --auth-url http://localhost:3000/login \
  --auth-email test@example.com \
  --auth-password s3cret
```

Via Config File

```
{
  "auth": {
    "strategy": "form",
    "loginUrl": "/login",
    "fields": {
      "email": "test@example.com",
      "password": "secret"
    },
    "submitSelector": "button[type=submit]"
  }
}
```

Trial / Guest Mode

If your app supports unauthenticated access (e.g., trial mode with localStorage-backed data), toetsLib detects this automatically. The analyzer identifies:

- localStorage usage for data persistence
- Conditional rendering based on `isAuthenticated`
- Feature restrictions for unauthenticated users

No credentials needed for trial-mode testing.

Security: Avoid committing real credentials to `.toetslib.json`. Use CLI flags or environment variables for sensitive data.

8. Generated Output

After running `toetslib generate`, your output directory contains:

```
e2e/ ├── playwright.config.ts # Auto-configured for your project ├── pages/ # Page Object Models │ ├── landing.page.ts │ ├── payroll.page.ts │ ├── employee-form.page.ts │ └── ... ├── fixtures/ # Test data and setup │ ├── base.fixture.ts # Extended Playwright fixtures │ ├── test-data.ts # Mock form data (context-aware) │ └── auth.setup.ts # Login flow (if auth configured) ├── tests/ # Test spec files │ ├── navigation.spec.ts │ ├── landing.spec.ts │ ├── employee-create.spec.ts │ └── ... ├── DIAGNOSTIC.md # Human-readable fix instructions └── .toetslib-report.json # Machine-readable results
```

Page Objects

Each page gets a Page Object Model with typed selectors and helper methods. Selectors are prioritized for stability:

1. `page.getByTestId()` — data-testid attributes (most stable)
2. `page.getByRole()` — ARIA roles
3. `page.getByLabel()` — form labels
4. `page.getByText()` — visible text
5. `page.locator()` — CSS selectors (last resort)

Test Categories

Category	What It Tests
Smoke	Page loads, key elements visible, no console errors
Happy path	Core user flows end-to-end (create, edit, submit)
Validation	Form errors on bad/missing input
Edge cases	Empty states, loading states, boundary conditions
Responsive	Layout at mobile (375px), tablet (768px), desktop (1280px)
Navigation	All links work, routing correct, mobile menu

Test Data

The `test-data.ts` fixture generates context-aware test data based on detected form fields. For South African apps, this includes SA ID numbers, ZA phone formats, and local addresses.

9. Diagnostic Reports

Instead of auto-fixing failures, toetsLib produces a diagnostic report that tells your team exactly what's wrong and how to fix it.

DIAGNOSTIC.md

A human-readable report with:

- Summary table (total, passed, failed, pass rate)
- Failures grouped by category
- Step-by-step fix instructions per failure
- Actionable recommendations

Failure Categories

Category	What It Means	Typical Fix
Selector	Element not found in DOM	Add `data-testid` attributes, update locator to use `getByRole`
Timeout	Element didn't appear in time	Check if backend APIs are running, add proper wait conditions
Assertion	Content doesn't match expected	Check formatting (currency, dates), use flexible matchers
Navigation	Page failed to load/redirect	Check route config, auth requirements, HTTPS settings

.toetslib-report.json

Machine-readable report for CI/CD integration:

```
{
  "summary": {
    "total": 45,
    "passed": 42,
    "failed": 3,
    "passRate": "93%"
  },
  "failures": [
    {
      "test": "shows validation error when email is empty
      "file": "employee-create.spec.ts",
      "category": "selector",
      "error": "locator('.error-msg') resolved to 0 eleme
      "fix": "Add data-testid to error message element...
    }
  ],
  "recommendations": [...]
}
```


10. Supported Frameworks

Framework	Detection	Notes
React + Vite	`vite` + `@vitejs/plugin-react`	Full support including workspace monorepos, HTTPS
Create React App	`react-scripts`	Detected from scripts.start
Next.js	`next`	App Router and Pages Router

Route Detection Patterns

Pattern	Supported
`<Route path="..." element={...} />`	Yes
`createBrowserRouter([...])`	Yes
Route arrays mapped with `.map()`	Yes
Route constants in separate files	Yes
File-based routing (Next.js)	Yes
Dynamic route params (`/user/:id`)	Yes

11. Real-World Example: Lefa Pay

Lefa Pay is a South African payroll management SPA built with React + Vite. Here's how toetsLib analyzes it:

Ingest Results

Field	Detected
Framework	react-vite
Package manager	npm (workspaces)
Dev command	npm run dev
Dev port	57760
HTTPS	Yes (ASP.NET dev certs)
Workspace root	ejm-lefa/src/

Analysis Results

Metric	Count
Routes	13 (8 public, 5 auth-required)
Page components	9 with child component trees
Forms	6 (employee: 30 fields, company: 17 fields, landing: 4 fields)
API endpoints	109
Auth type	Context-based with trial mode

Crawl Results

Metric	Count
Pages crawled	8
Screenshots captured	24 (8 pages × 3 viewports)
Interactive elements	72
Navigation edges	13

Running It

```
toetslib generate ./ejm-lefa/src/pay.Client \
  --output ./ejm-lefa/src/e2e \
  --verbose
```

12. CI/CD Integration

GitHub Actions

```yaml
name: E2E Tests
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '22'

      - name: Install toetsLib
        run: |
          cd toetslib && npm install && npx turbo build

      - name: Generate tests
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KE
        run: |
          toetslib generate . --output ./e2e --no-validat

      - name: Run tests
        run: |
          cd e2e && npx playwright install chromium
          npx playwright test

      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: test-results
          path: |
            e2e/DIAGNOSTIC.md
            e2e/.toetslib-report.json
            e2e/test-results/
```

Quality Gate

```
# Fail CI if pass rate is below threshold
PASS_RATE=$(jq -r '.summary.passRate' e2e/.toetslib-repor
if [ "$PASS_RATE" -lt 80 ]; then
  echo "Pass rate $PASS_RATE% is below 80% threshold"
  exit 1
fi
```

13. Troubleshooting

"Framework not detected"

Ensure `package.json` lists React and a bundler (vite, react-scripts, next) as dependencies. Use `.toetslib.json` to manually specify `devCommand` and `devPort`.

"Dev server failed to start"

- Check that `npm run dev` works when run manually
- For monorepos, toetsLib auto-detects the workspace root and installs from there
- Check if the port is already in use: `lsof -ti :PORT`

"No routes found"

Run `toetslib analyze ./my-app --json --verbose` to see what the parser finds. Routes may use patterns the AST parser doesn't recognize yet.

"ANTHROPIC_API_KEY not set"

The `generate` command requires an API key. Set it: `export ANTHROPIC_API_KEY="sk-ant-..."`

"Tests fail after generation"

Check `DIAGNOSTIC.md` for categorized failure analysis with fix instructions. Most common issues are selector mismatches (add `data-testid` attributes) or missing backend APIs (run the backend or mock responses).

14. Architecture

toetsLib is a Turborepo monorepo with focused packages:

Package	Purpose
`@toetslib/cli`	CLI entry point with 4 commands (commander)
`@toetslib/core`	Pipeline orchestrator, AppMap types (15 interfaces)
`@toetslib/shared`	Logger, file utilities, Claude API client with prompt caching
`@toetslib/ingest`	Framework detection, dev server management
`@toetslib/analyzer`	Babel AST-based static code analysis
`@toetslib/crawler`	Headless Playwright browser crawling
`@toetslib/generator`	3-pass AI test generation via Claude API
`@toetslib/validator`	Test runner and diagnostic report writer

AppMap — The Central Data Model

A single TypeScript object flows through all 5 pipeline stages. Each stage enriches it with its findings:

- **Stage 1:** `project` (framework, port, dev command)
- **Stage 2:** `routes`, `forms`, `apiCalls`, `auth`, `components`
- **Stage 3:** `pages`, `interactions`, `screenshots`, `navigationGraph`
- **Stage 4:** `testPlan` + files written to disk
- **Stage 5:** `results` + diagnostic report

AI Strategy

The LLM client uses Anthropic's prompt caching (`cache_control: { type: "ephemeral" }`) on the system prompt. Within the 5-minute

TTL, all per-page generation calls reuse the cached system prompt, reducing cost and latency.

Screenshots are sent as images to the AI during page object generation, so the model can "see" the actual page layout while writing selectors and test code.